

Vorlesung: Grundlagen der Computergrafik

Kapitel 2: 2D-Computergrafik

Vorlesung im Sommersemester 2026

Prof. Dr. Stephan Diehl, Informatik, Universität Trier



Organisatorisches

- Nächste Woche 27.4 Übung

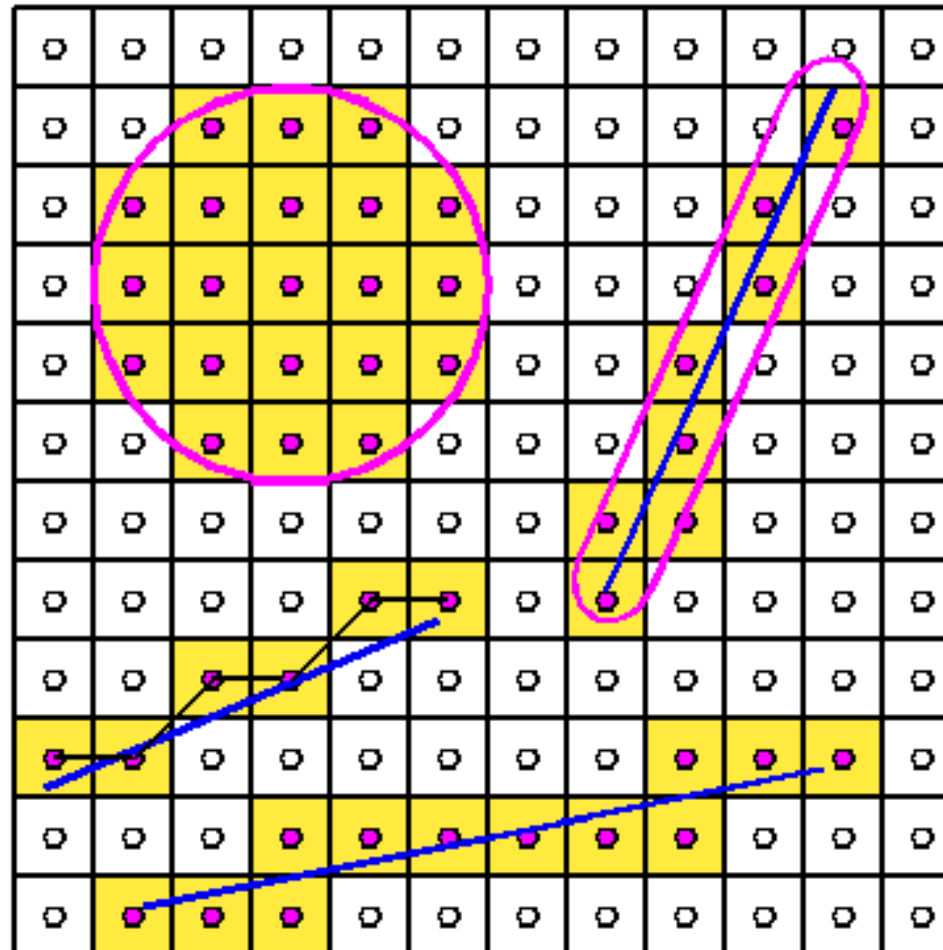
RASTER DISPLAY

Which PIXELS should be turned on ?



Sampling
the given shape
at pixel centers

Closest matching
path through
pixel centers



Sampling an
idealized shape
around line

Turning on pixels
touched by line

Kapitelübersicht

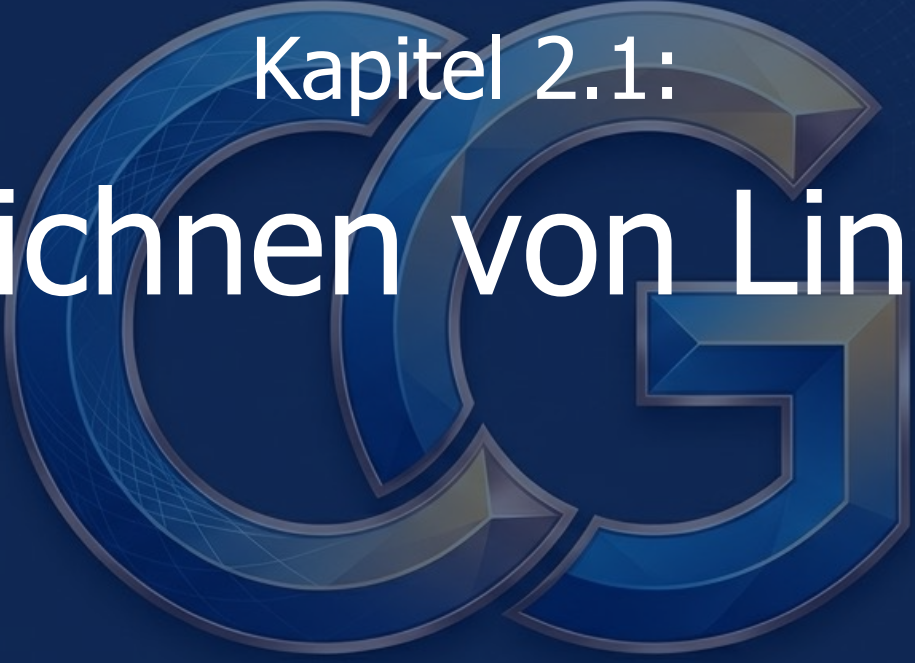
- [Kapitel 2.1:](#) Zeichnen von Linien
- [Kapitel 2.2:](#) Füllen von Polygonen
- [Kapitel 2.3:](#) SVG

28.04.2025	Rastering, Zeichnen von Linien
05.05.2025	Transformationen + Projektionen
12.05.2025	SVG (Linien, Transformation, Animation)

Vorlesung: Grundlagen der Computergrafik

Kapitel 2: 2D-Computergrafik

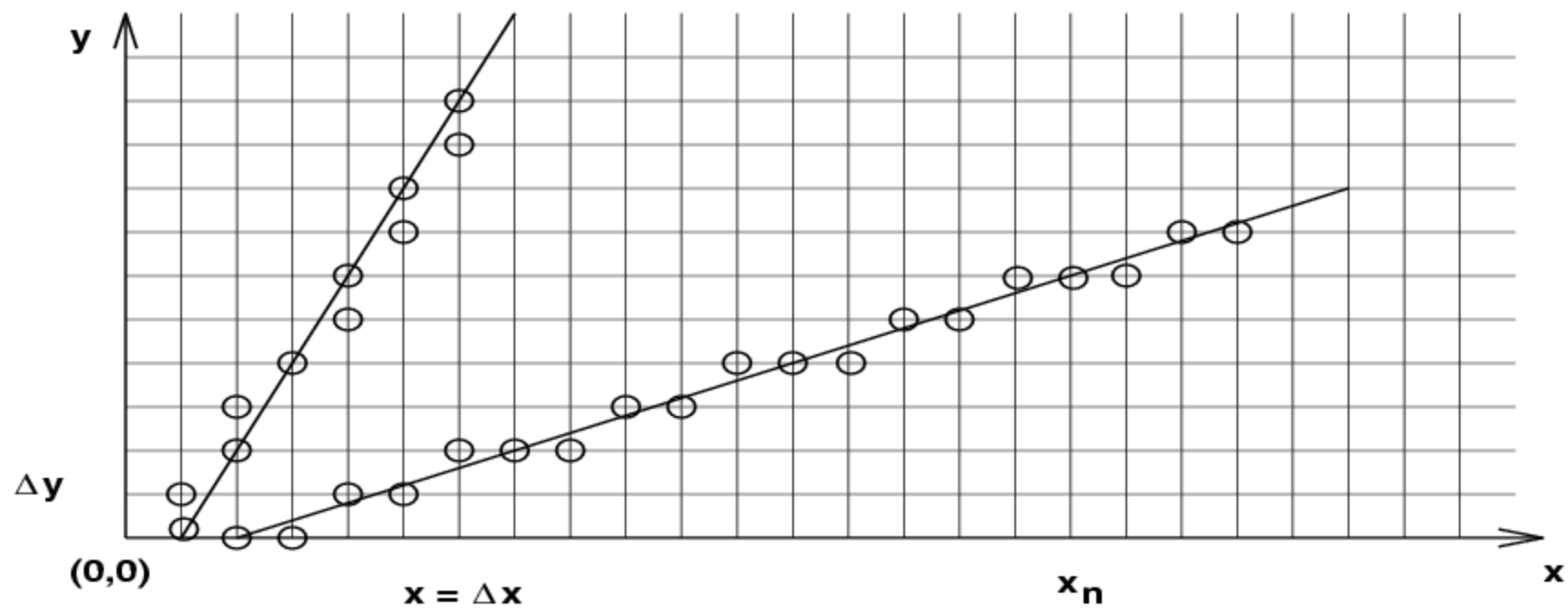
Kapitel 2.1: Zeichnen von Linien

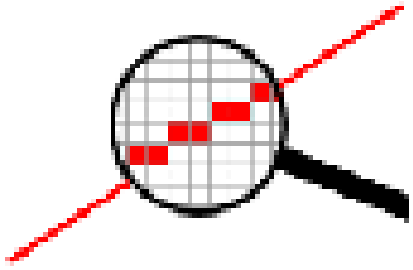


Vorlesung im Sommersemester 2026

Prof. Dr. Stephan Diehl, Informatik, Universität Trier







Rasterung

- Wie stellt man Linien auf einem Raster dar ?
- Darstellung von Geraden:
 - Implizit: $c_0 \cdot x + c_1 \cdot y + c_2 = 0$ mit Steigung c_0/c_1
 - Explizit: $y = a \cdot x + b$ mit Steigung a
 - Parameterform:
$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} + t \cdot \begin{pmatrix} a_x \\ a_y \end{pmatrix}$$

Rasterung von Linien

- **1. Ansatz:** Wir wollen die Gerade $a \cdot x + b$ an den ganzzahligen Rasterpalten x_0 bis x_n zeichnen:

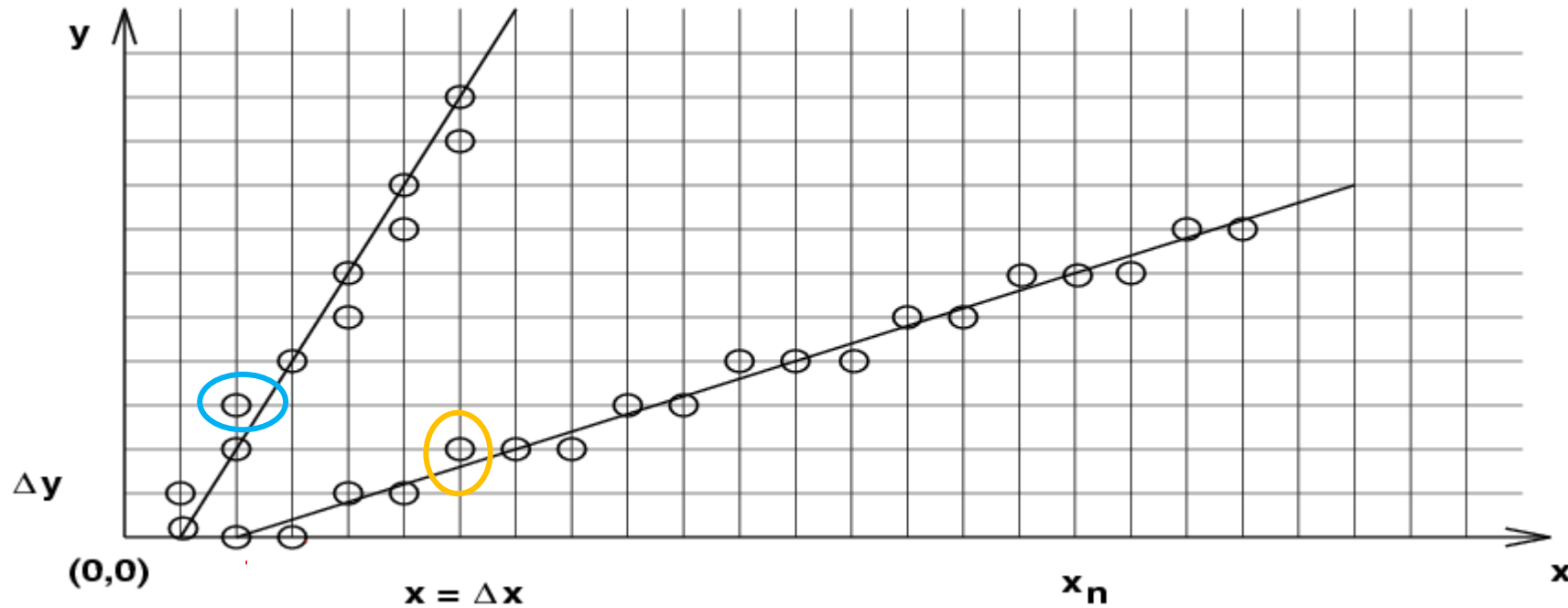
$$y_i = \text{round}(a \cdot x_i + b)$$

Reellzahlige Multiplikation

Reellzahlige Addition

Runden

Problem: zu rechenaufwendige Operationen



Im Folgenden: x_i, y_i Koordinaten von Gitterpunkten

$\tilde{x}_i, \tilde{y}_i$ Koordinaten von Schnittpunkten der Geraden

2. Ansatz:

- Iterativ: Berechne nächsten Punkt auf Basis des vorangegangenen Punktes
- Ziel: reellzahlige Multiplikation eliminieren

Rasterung von Linien

- **2. Ansatz:** Berechne nächsten Punkt auf Basis des vorangegangenen Punktes
 - Iterativ
 - Ziel: reellzahlige Multiplikation eliminieren

1. Fall: $|a| < 1$ (ein y-Wert pro x-Wert)

$$\tilde{y}_i = a \cdot x_i + b$$

$$\text{Schritt: } x_{i+1} = x_i + \Delta x$$

$$\tilde{y}_{i+1} = a \cdot x_{i+1} + b = a \cdot (x_i + \Delta x) + b$$

$$= a \cdot x_i + b + a \cdot \Delta x = \tilde{y}_i + a \cdot \Delta x$$

$$\text{für } \Delta x = 1 \\ = \tilde{y}_i + a$$

$$y_{i+1} = \text{round}(\tilde{y}_{i+1})$$

Rasterung von Linien

2. Fall: $|a| > 1$

Schritt: $y_{i+1} = y_i + \Delta y$

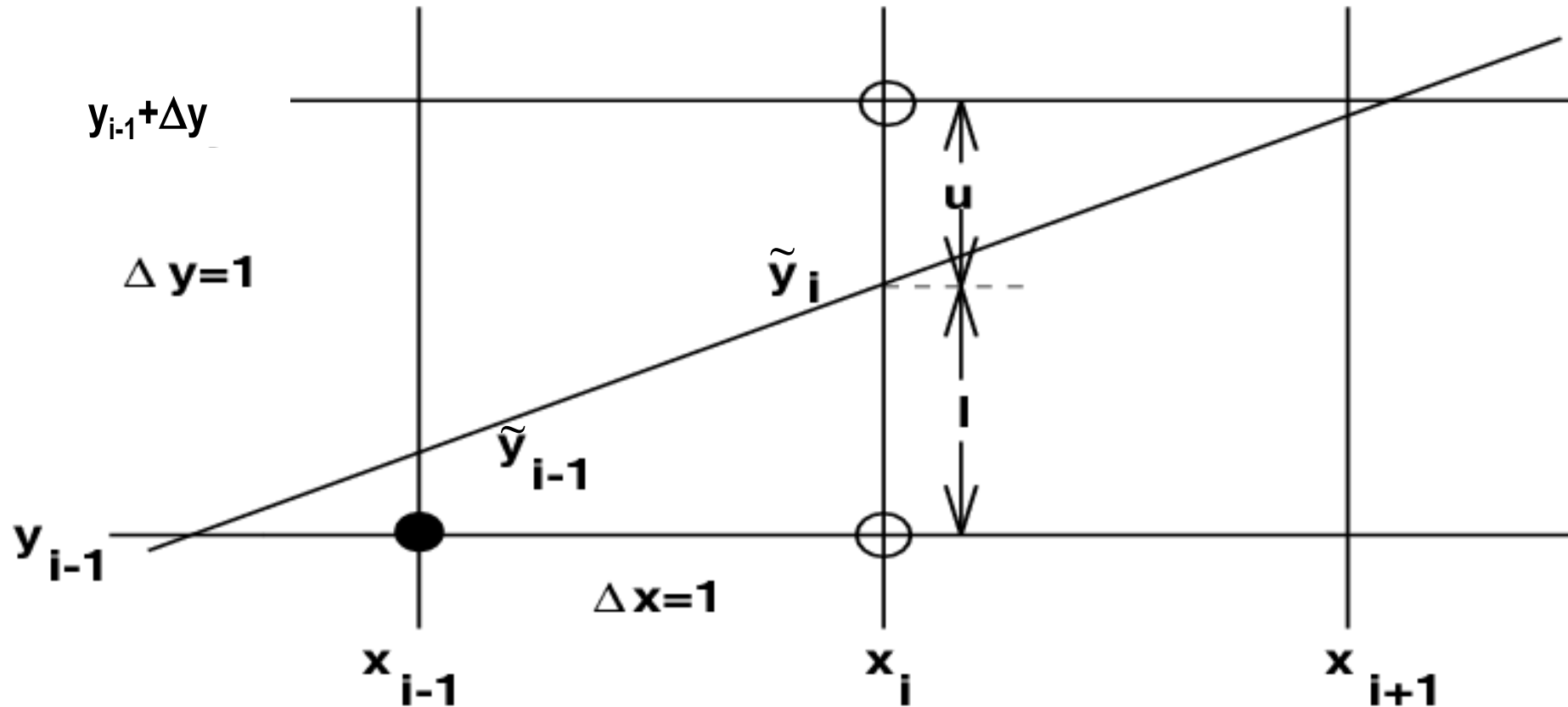
$$y_i = a \cdot \tilde{x}_i + b \quad \Rightarrow \quad \tilde{x}_i = \frac{y_i - b}{a}$$

$$\tilde{x}_{i+1} = \frac{y_{i+1} - b}{a} = \frac{y_i + \Delta y - b}{a} = \tilde{x}_i + \frac{\Delta y}{a} \quad \text{für } \Delta y = 1 \quad = \tilde{x}_i + \frac{1}{a}$$

$$x_{i+1} = \text{round}(\tilde{x}_{i+1})$$

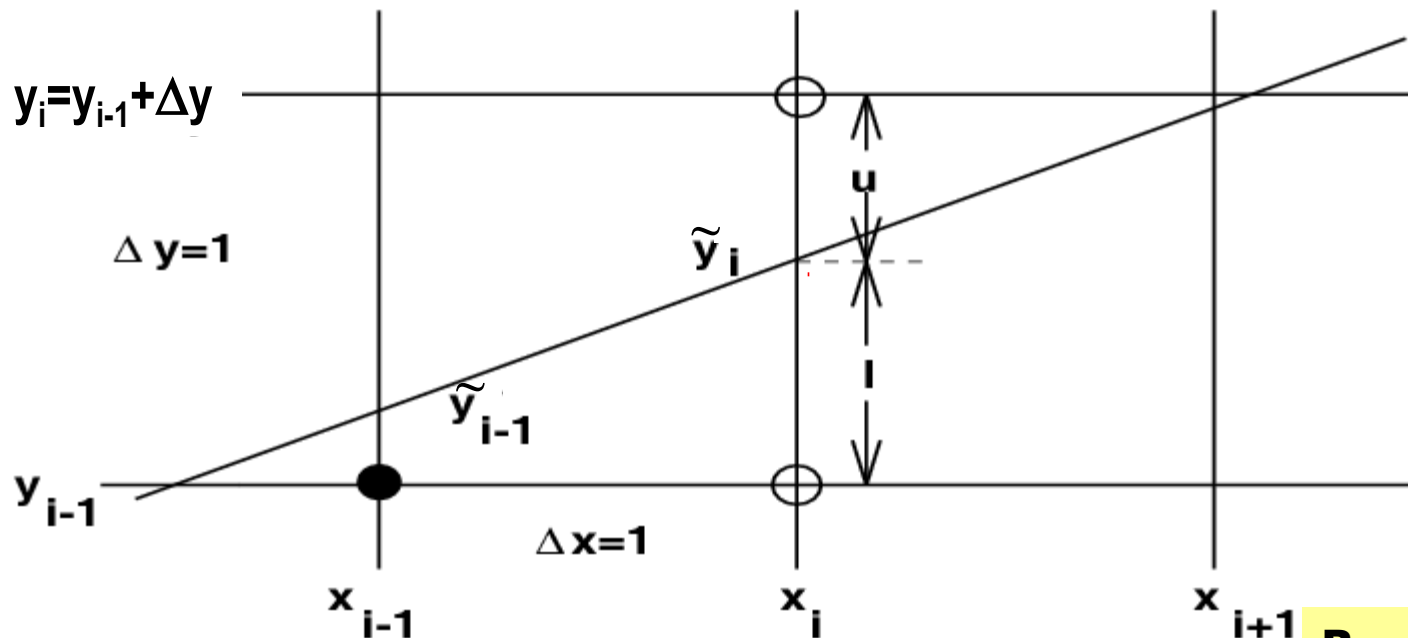
Problem: immer noch
reellzahlige Addition und
Runden

Rasterung von Linien



Rasterung von Linien

- Der Bresenham-Algorithmus
 - Iterativ, nur Integerarithmetik
 - Distanz jedes Punktes zur Gerade minimieren



Bemerkung:

Wir betrachten hier nur den Fall: $0 < a < \Delta y / \Delta x$
D.h. die Linie verläuft im ersten Oktanten.

$$l = \tilde{y}_i - y_{i-1} \quad u = y_{i-1} + \Delta y - \tilde{y}_i$$

3. Ansatz:

- Bresenham: Entscheidungsvariable; Ziel: nur Integerarithmetik iterativ

2. Ansatz, erster Fall: $\tilde{y}_{i+1} = \tilde{y}_i + a \cdot \Delta x$

Zusammengefasst:

$$l = \tilde{y}_i - y_{i-1}$$
$$u = y_{i-1} + \Delta y - \tilde{y}_i$$

- Entscheidungsvariable $d_i = l - u$
 - für $l < u \Rightarrow d_i < 0$ wähle $y_i = y_{i-1}$
 - für $l > u \Rightarrow d_i > 0$ wähle $y_i = y_{i-1} + \Delta y$

} Minimiere Abstand zur Geraden

$$d_i = l - u = (\tilde{y}_i - y_{i-1}) - (y_{i-1} + \Delta y - \tilde{y}_i)$$
$$= 2\tilde{y}_i - 2y_{i-1} - \Delta y$$

Ziel: d_{i+1} aus d_i berechnen, also iterativ

- 1. Fall: $d_i > 0 \quad \Rightarrow \quad y_i = y_{i-1} + \Delta y$

$$\begin{aligned}
 d_{i+1} &= 2\tilde{y}_{i+1} - 2y_i - \Delta y \\
 &= 2(\tilde{y}_i + \Delta x a) - 2(y_{i-1} + \Delta y) - \Delta y \\
 &= 2\tilde{y}_i + 2a \Delta x - 2y_{i-1} - 2\Delta y - \Delta y \\
 &= 2\tilde{y}_i - 2y_{i-1} - \Delta y + 2a \Delta x - 2\Delta y \\
 &\quad \underbrace{\hspace{1.5cm}}_{=d_i} = d_i + 2a \Delta x - 2\Delta y
 \end{aligned}$$

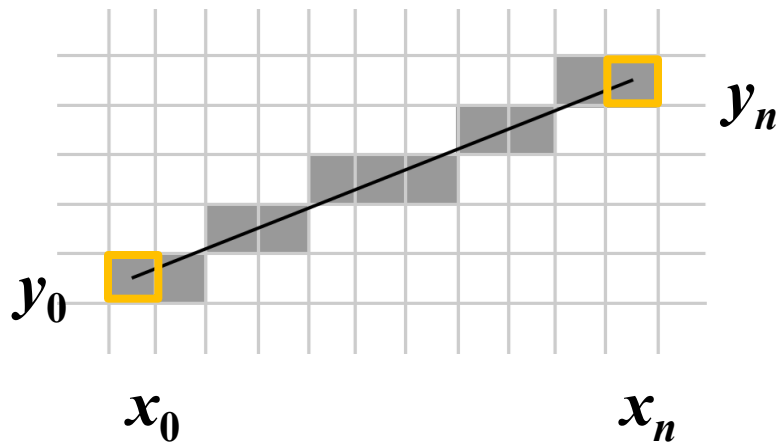
- 2. Fall: $d_i < 0 \quad \Rightarrow \quad y_i = y_{i-1}$

$$\begin{aligned}
 d_{i+1} &= 2\tilde{y}_{i+1} - 2y_i - \Delta y \\
 &= 2(\tilde{y}_i + \Delta x a) - 2y_{i-1} - \Delta y \\
 &= 2\tilde{y}_i + 2a \Delta x - 2y_{i-1} - \Delta y = d_i + 2a \Delta x
 \end{aligned}$$

Problem: a ist reelle Zahl

- Eliminieren der reellen Zahl a

Sei a gegeben durch $a = dy/dx$ mit $dy = y_n - y_0$ und $dx = x_n - x_0$ und (x_0, y_0) und (x_n, y_n) Anfangs- bzw. Endpunkt. Außerdem sind dx und dy positiv, da wir nur den Fall $0 < a < 1$ betrachten (1. Oktant). Statt der Entscheidungsvariable d_i können wir auch $d_i' = dx \cdot d_i$ verwenden und iterativ berechnen:



- Eliminieren der reellen Zahl a

Sei a gegeben durch $a=dy/dx$ mit $dy=y_n-y_0$ und $dx=x_n-x_0$ und (x_0,y_0) und (x_n,y_n) Anfangs- bzw. Endpunkt. Außerdem sind dx und dy positiv, da wir nur den Fall $0<a<1$ betrachten (1. Oktant). Statt der Entscheidungsvariable d_i können wir auch $d_i' = dx \cdot d_i$ verwenden und iterativ berechnen:

1. Fall: $d_i > 0$

$$\begin{aligned} d_{i+1}' &= dx \cdot d_{i+1} = dx (d_i + 2a \Delta x - 2\Delta y) = d_i dx + 2a \Delta x dx - 2 \Delta y dx \\ &= d_i' + 2 \Delta x (dy/dx) dx - 2 \Delta y dx = d_i' + 2\Delta x dy - 2 \Delta y dx \end{aligned}$$

2. Fall: $d_i < 0$

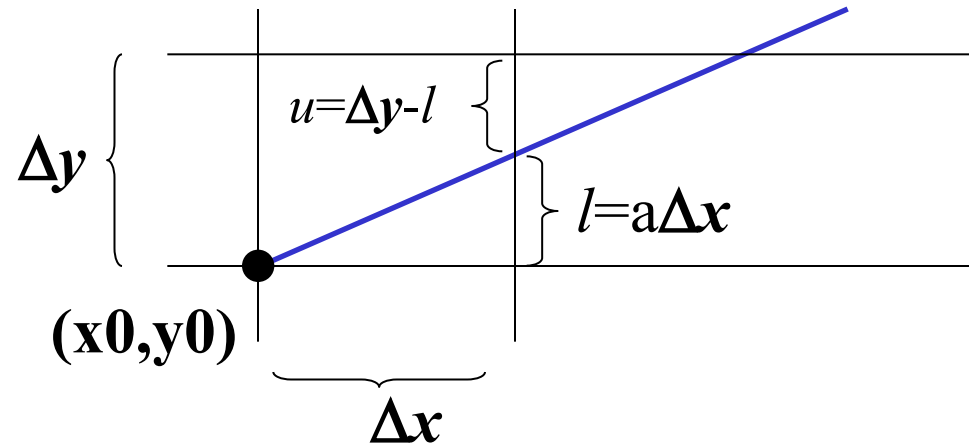
$$\begin{aligned} d_{i+1}' &= dx \cdot d_{i+1} = dx (d_i + 2a \Delta x) = d_i dx + 2a \Delta x dx \\ &= d_i' + 2 \Delta x (dy/dx) dx = d_i' + 2 \Delta x dy \end{aligned}$$

Beobachtung: d_i' und d_{i+1}' sind ganzzahlig !!!

Bresenham-Algorithmus für den 1. Oktanten

```
bresenham( (x0,y0) , (xn,yn) )  
{ dx=xn-x0;  
  dy=yn-y0;  
  d' = 2dy - dx;    // Startwert  
  y = y0;  
  for(x=x0; x<=xn ; x++)  
  { drawpoint(x,y);  
    if (d'<0)  
    { d' = d'+2*dy;  
    }  
    else  
    { d' = d' + 2(dy-dx);  
      y=y+1;  
    }  
  }  
}
```

Startwert für d'

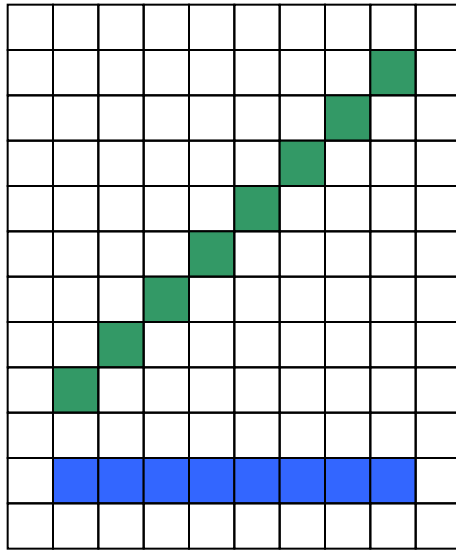


$$d = l - u = a \Delta x - (\Delta y - a \Delta x) = 2a\Delta x - \Delta y = 2\Delta x(dy/dx) - \Delta y$$

$$d' = dx \, d = dx (2\Delta x dy/dx - \Delta y) = 2\Delta x dy - \Delta y dx$$

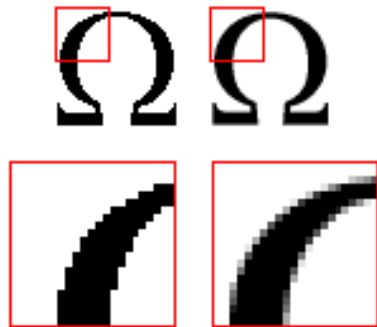
Rasterung von Linien

- Es gibt viele andere Algorithmen zum Zeichnen von Linien.



Linien mit unterschiedlicher Steigung erscheinen unterschiedlich hell. Die grüne Linie ist $\sqrt{2}$ -mal länger als die blaue.

Eine Lösung: Intensität mit der Steigung erhöhen.



Treppeneffekt bei diagonalen oder gekrümmten Linien.

Lösung: Antialiasing Verfahren

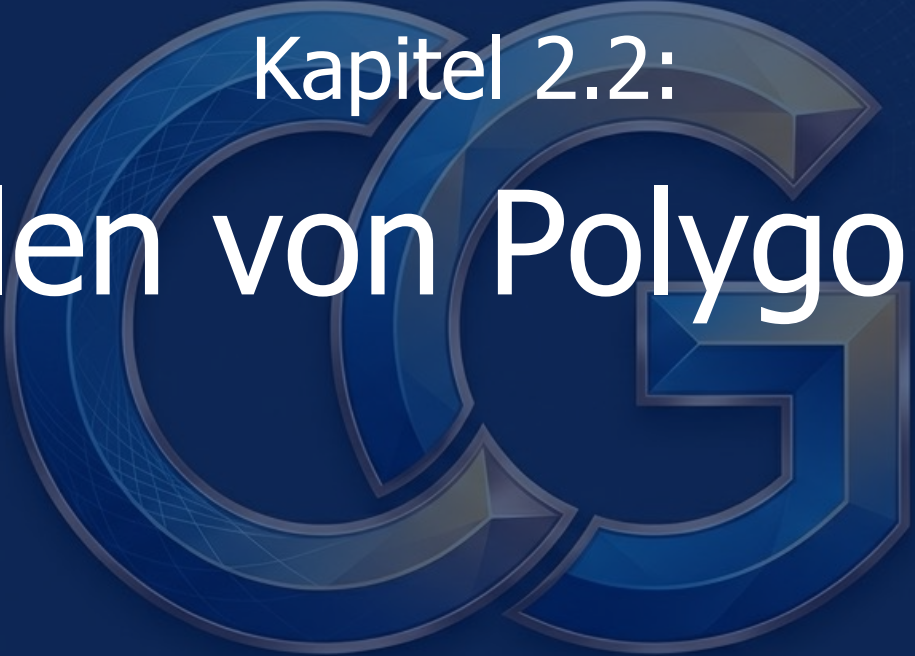
Rasterung von Kreisen

- Implizite Darstellung: $x^2 + y^2 - R^2 = 0$
- Explizit: $y = \pm \sqrt{R^2 - x^2} \quad 0 \leq x \leq R$
- Ausgehend von der expliziten Darstellung kann man auch iterative Algorithmen mit Entscheidungsvariablen ähnlich zu dem vorherigen Algorithmus herleiten.

Vorlesung: Grundlagen der Computergrafik

Kapitel 2: 2D-Computergrafik

Kapitel 2.2: Füllen von Polygonen



Vorlesung im Sommersemester 2026

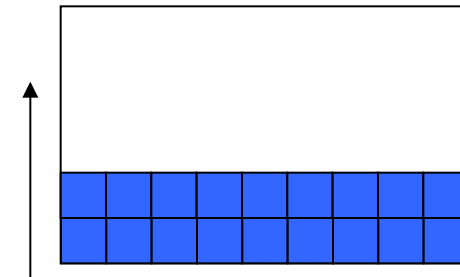
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



Der Scan-Line Algorithmus

- Scan-Line Verfahren rastern Zeile für Zeile eines Polygons innerhalb einer vorgegebenen Begrenzungskurve. Für ein Rechteck ist dieser Algorithmus sehr einfach darzustellen

```
for y = ymin to ymax
{ for x = xmin to xmax
    { drawpoint(x,y,color)
    }
}
```



Scan-Line Algorithmus für Polygone

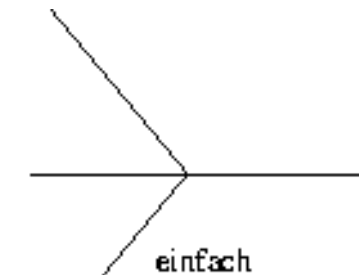
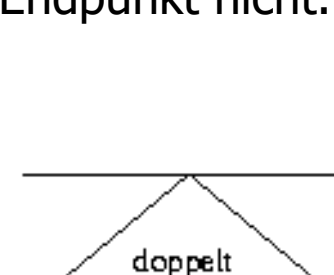
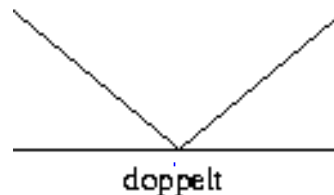
1. Sortiere die Polygonkanten nach dem kleinsten (y-Wert)
2. Beginne mit unterer Scan Linie (vom y_{min} zu y_{max})
3. Finde Schnittpunkte der Scan Linie mit *allen* Kanten des Polygons
4. Sortiere Schnittpunkte nach zunehmendem x-Wert
5. Fülle die Pixel im Inneren des Polygons unter Verwendung der Paritätsregel auf.

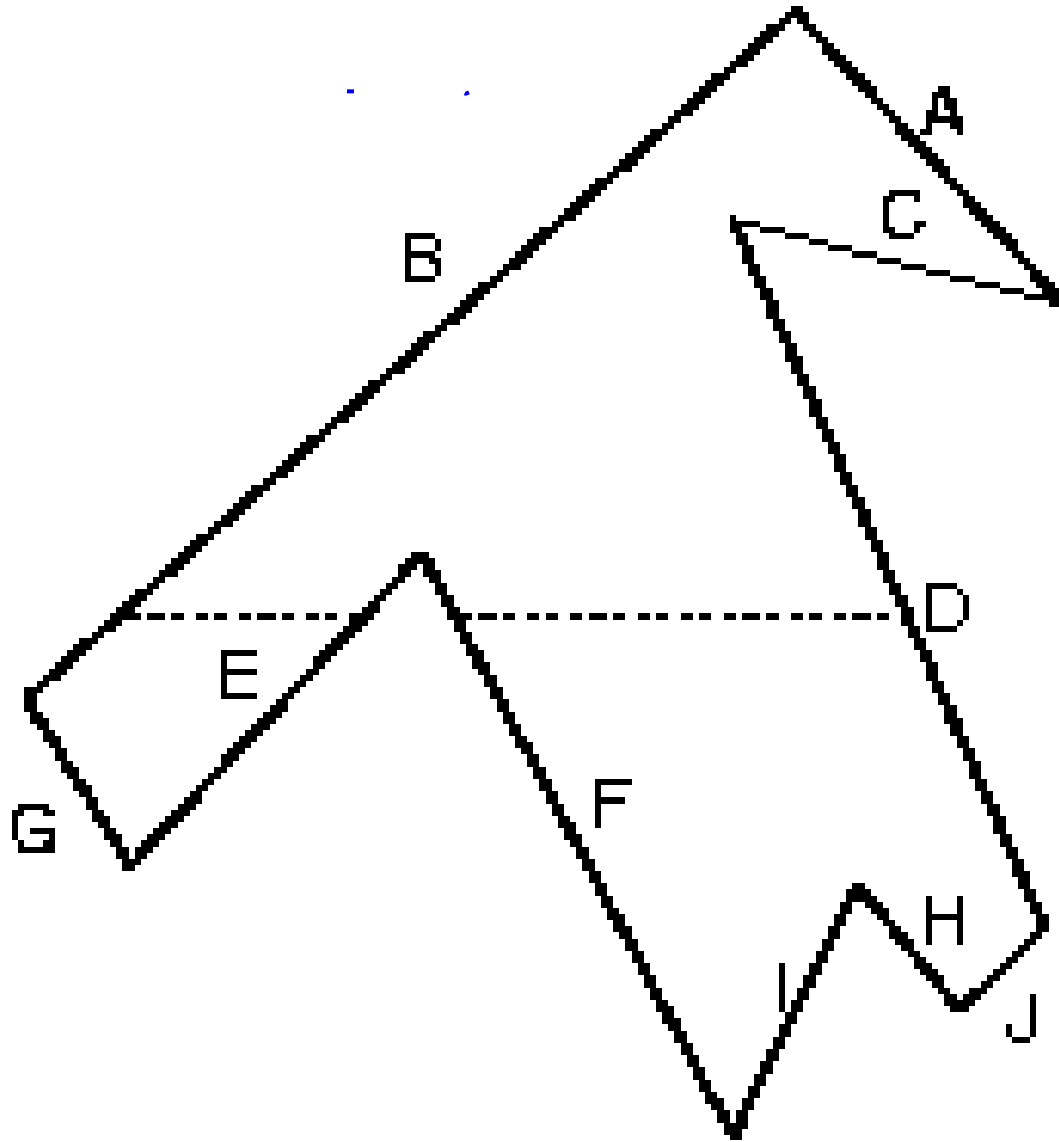
Paritätsregel:

Bei gerader Parität werden Pixel nicht gezeichnet, bei ungerader Parität werden sie gezeichnet. Man beginnt mit gerader Parität. Jede Schnittstelle invertiert die Parität.

Sonderfälle:

- Horizontale Kanten werden ignoriert
- Werden die Zeilen nach aufsteigenden y-Koordinaten bearbeitet, werden an den Polygoneckpunkten die Kanten berücksichtigt, die dort einen y-minimalen Endpunkt haben, diejenigen mit y-maximalem Endpunkt nicht.





1. Sortiere die Polygonkanten nach dem kleinsten (y-Wert) I,F,H,J,D,...
2. Beginne mit unterer Scan Linie (vom ymin zu ymax)
3. Finde Schnittpunkte der Scan Linie mit *allen* Kanten des Polygons
4. Sortiere Schnittpunkte nach zunehmendem x-Wert
5. Fülle die Pixel im Inneren des Polygons unter Verwendung der Paritätsregel auf.